UNIVERSIDAD POLITÉCNICA SALESIANA ECUADOR

Práctica 4

Patrones de Diseño de APIs



Elaborado por:

Carlos Adrián Espinoza

Optimización de consultas GraphQL con DataLoader

Informe de práctica — Patrones de Diseño de APIs

Estudiante	Carlos Adrián Espinoza
Asignatura	Patrones de Diseño de APIs
Repositorio	github.com/AdrianEspinoza92/PatronesDisenoApis-Practica-4
Fecha	Septiembre de 2026

1. Descripción de la actividad

Implementé una API GraphQL con enfoque Schema-First para consultar clientes y sus facturas. Primero construí una versión ingenua del resolver con el propósito de evidenciar el problema N+1; después incorporé DataLoader para agrupar las consultas de facturas y mantener un caché aislado por solicitud.

El objetivo técnico fue demostrar que la optimización no modifica el contrato ni la respuesta de la API, pero sí reduce la cantidad de accesos a datos cuando se resuelven relaciones anidadas.

2. Desarrollo de la práctica

1. Definí los tipos Cliente y Factura, junto con la consulta clientes, en schema.graphql.
2. Implementé resolvers ingenuos que consultan las facturas una vez por cada cliente.
3. Creé un DataLoader por petición para consolidar identificadores y resolverlos en una sola operación batch.
4. Conservé el orden de las respuestas, representé clientes sin facturas y verifiqué el comportamiento del caché.
5. Ejecuté pruebas automatizadas y una demostración comparativa con la misma consulta GraphQL.

3. Código fuente relevante

```
// dataloaders.js
const invoiceLoader = new DataLoader(async (clientIds) => {
  const invoices = await database.findInvoicesByClientIds(clientIds);
  return clientIds.map(id => invoices.filter(item => item.clientId === id));
});
```

La instancia del cargador se crea dentro del contexto de cada petición. Con esta decisión evité que el caché mezclara información entre usuarios o solicitudes independientes.

4. Capturas de pantallas funcionales



Figura 1. Ejecución real de las cinco pruebas automatizadas del repositorio.

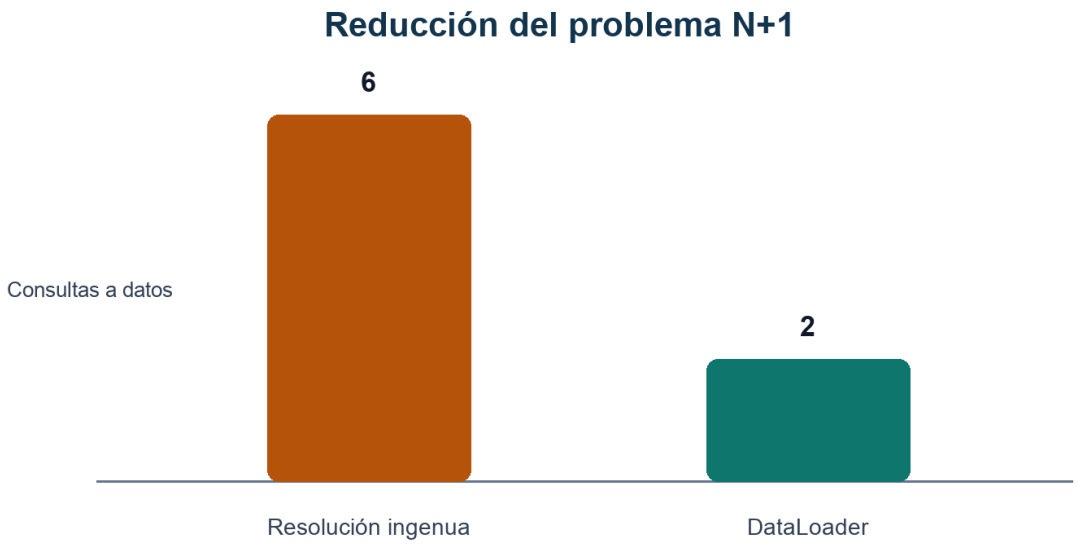


Figura 2. Comparación de consultas a datos para la misma respuesta GraphQL.

5. Resultados y análisis

Implementación ingenua	6 consultas: 1 para clientes y 5 para facturas.
Implementación optimizada	2 consultas: 1 para clientes y 1 consulta batch de facturas.
Reducción	4 consultas menos, equivalente a 66,7 %.
Pruebas	5 aprobadas, 0 fallidas, duración aproximada de 224 ms.

Los datos confirmaron el patrón N+1: al aumentar el número de clientes, el resolver ingenuo agrega una consulta por cada elemento. Con DataLoader, el costo se mantiene en dos operaciones para esta consulta. También comprobé que si el cliente no solicita el campo facturas, no se ejecuta la consulta correspondiente.

6. Conclusiones

- Comprobé que DataLoader elimina el crecimiento lineal de consultas asociado al problema N+1 sin cambiar el esquema GraphQL ni la estructura de la respuesta.
- Determiné que el alcance por petición del cargador es indispensable: permite aprovechar el caché dentro de una operación y evita compartir datos entre solicitudes diferentes.

7. Bibliografía

- GraphQL Foundation. (2026). Schemas and types. https://graphql.org/learn/schema/
- Meta Open Source. (2026). DataLoader: Data loading and caching utility. https://github.com/graphql/dataloader
- GraphQL Foundation. (2026). Execution. https://graphql.org/learn/execution/